

EPIC 06 — Database & Distributed Data Design

Jira Task 01: Database Design & Database-per-Service Architecture

Sprint: Sprint 12 (21/9 – 4/10)

Priority: Critical **Effort:** 8 Hours

Objective: Understand how databases should be designed and owned independently by microservices, and document the resulting architecture.

Contents

- 1. Relational database fundamentals
- 2. Why database-per-service
- 3–5. Schema design: User, Order, Payment databases
- 6. ER diagrams
- 7. Data ownership
- 8. Removing direct database dependency
- 9. Service-to-service communication
- 10. Architecture summary

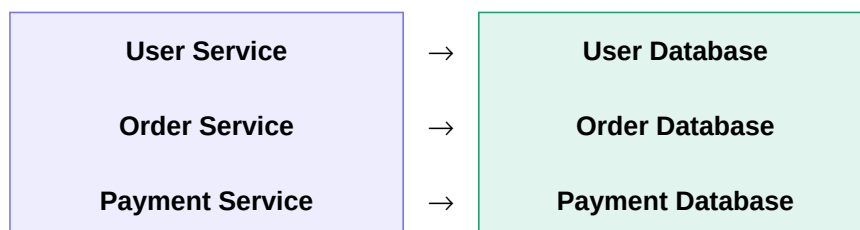
Step 1 — Relational database fundamentals

A relational database organizes data into **tables** — similar to linked spreadsheets. Each table stores one type of entity (Users, Orders, Payments). Rows are individual records; columns are the fields describing each record.

Concept	Meaning
Primary key	A column that uniquely identifies each row. No two rows share the same value.
Foreign key	A column in one table that references the primary key of another table, creating a link between them.
Relationships	How tables connect: one-to-many (one user, many orders), many-to-many, or one-to-one.
Constraints	Rules enforced by the database, e.g. NOT NULL, UNIQUE, FOREIGN KEY.
Indexes	A lookup structure that speeds up searches on specific columns.
Normalization	Structuring data to avoid repetition — store each fact once and reference it.
Denormalization	Deliberately duplicating some data to speed up reads, at the cost of consistency risk.

Step 2 — Database-per-service

In a microservices architecture, each service owns and manages its own database. No service is allowed to directly query another service's database.



Why this matters: each service can change its own schema without breaking others, scale independently, use the best-fit database technology, and keep failures isolated — if the Payment Database goes down, User Service and Order Service keep working.

Steps 3–5 — Database schemas

Each database below was created and verified directly in MySQL Workbench, matching each service's actual entity classes.

3. User database — users

Field	Type	Null	Key
id	BIGINT	NO	PRI
name	VARCHAR(100)	NO	
email	VARCHAR(150)	NO	UNI
phone	VARCHAR(20)	YES	
status	VARCHAR(20)	YES	
created_at	TIMESTAMP	YES	
updated_at	TIMESTAMP	YES	

Table: users — confirmed present and matching via user-service (running app).

4a. Order database — orders

Field	Type	Null	Key
id	BIGINT	NO	PRI
user_id	BIGINT	YES	
product_id	BIGINT	YES	
product_name	VARCHAR(255)	YES	
quantity	INT	YES	
amount	DECIMAL(38,2)	YES	
status	VARCHAR(255)	YES	
created_at	DATETIME(6)	YES	

Table: orders — auto-created by Hibernate from the Order entity; verified via DESCRIBE orders.

4b. Order database — order_items

Field	Type	Null	Key
id	BIGINT	NO	PRI
order_id	BIGINT	NO	MUL
product_name	VARCHAR(150)	NO	
quantity	INT	NO	
price	DECIMAL(10,2)	NO	

Table: order_items — created manually; order_id has a real foreign key to orders.id (same database).

5. Payment database — payments

Field	Type	Null	Key
id	BIGINT	NO	PRI
order_id	BIGINT	NO	
amount	DECIMAL(10,2)	NO	
payment_status	VARCHAR(20)	YES	
transaction_reference	VARCHAR(100)	YES	
created_at	TIMESTAMP	YES	

Table: payments — created fresh in payment_db. order_id is a plain reference, NOT a foreign key (different database).

Important design point: order_id and user_id fields that point across databases are never declared as SQL FOREIGN KEY constraints, because a real foreign key cannot span two separate databases. They are plain columns whose validity is guaranteed by the owning service's API, not by the database engine.

Step 6 — ER diagrams

Each database is diagrammed separately below (no cross-database diagram exists, by design — see Step 8).

USERS

USERS	
PK	id : BIGINT
	name : VARCHAR(100)
	email : VARCHAR(150) [UNIQUE]
	phone : VARCHAR(20)
	status : VARCHAR(20)
	created_at : TIMESTAMP
	updated_at : TIMESTAMP

ORDERS 1 ■■■■< ORDER_ITEMS (one order has many order items — real foreign key, same database)

ORDERS

ORDERS	
PK	id : BIGINT
	user_id : BIGINT [ref → User Service, not FK]
	product_id : BIGINT
	product_name : VARCHAR(255)
	quantity : INT
	amount : DECIMAL(38,2)
	status : VARCHAR(255)
	created_at : DATETIME

ORDER_ITEMS

ORDER_ITEMS	
PK	id : BIGINT
FK	order_id : BIGINT → orders.id
	product_name : VARCHAR(150)
	quantity : INT
	price : DECIMAL(10,2)

PAYMENTS

PAYMENTS	
PK	id : BIGINT
	order_id : BIGINT [ref → Order Service, not FK]
	amount : DECIMAL(10,2)
	payment_status : VARCHAR(20)
	transaction_reference : VARCHAR(100)
	created_at : TIMESTAMP

Step 7 — Data ownership

Service	Owns
User Service	users
Order Service	orders, order_items
Payment Service	payments

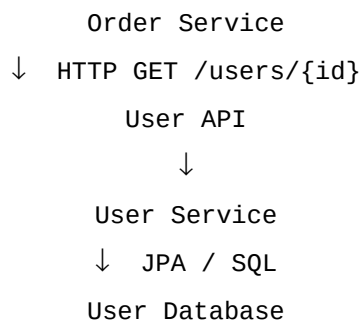
Only the owning service ever writes to its table. Other services may read the same data only by calling that service's API — never by connecting to its database directly.

Step 8 — Removing direct database dependency

Status: already satisfied by the existing codebase. order-service does not contain any SQL query, JPA repository, or entity that touches the users table. When order-service needs user information, it goes through `UserServiceClient`, an HTTP client class — not a database connection to `user_db`.

Step 9 — Service-to-service communication

When Order Service needs user data, the call travels through the network, not the database layer:



This is exactly the pattern already implemented and tested: order-service calls `UserServiceClient.getUserDetails(userId)`, which makes an HTTP request to User Service's `/users/{id}` endpoint. Only User Service ever touches the User Database.

Step 10 — Architecture summary

This microservices system follows the **database-per-service** pattern. Three independent databases exist — `user_db`, `order_db`, and `payment_db` — each owned exclusively by its corresponding service.

Design decisions:

- Each service's schema was designed around what that service alone needs to function, not shared conveniences.
- Cross-service references (`order.user_id`, `payment.order_id`) are stored as plain columns, never as SQL FOREIGN KEY constraints, since MySQL cannot enforce a foreign key across two databases.
- `order_items` uses a real foreign key to `orders.id` because both tables live inside the same `order_db` — referential integrity can be enforced there.
- Any service needing data owned by another service calls that service's REST API (e.g. `UserServiceClient` → `GET /users/{id}`) rather than querying its database.
- This isolation means each service can be deployed, scaled, migrated, or taken offline without directly breaking another service's data layer.

Deliverables checklist

Item	Status
Database schemas	Created — users, orders, order_items, payments (verified in MySQL Workbench)
ER diagrams	Included in Step 6 of this document
Database-per-service architecture	Documented in Step 2 and Step 10
Data ownership document	Included in Step 7
SQL scripts / migrations	See Appendix A
Architecture diagram	Included in Step 2 and Step 9
Git commit	Pending — commit schema scripts to the repository

Appendix A — SQL scripts

User database

```
CREATE DATABASE user_db;
USE user_db;

CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  phone VARCHAR(20),
  status VARCHAR(20) DEFAULT 'ACTIVE',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

Order database

```
CREATE DATABASE order_db;
USE order_db;

-- orders table auto-created by Hibernate from the Order entity

CREATE TABLE order_items (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_id BIGINT NOT NULL,
  product_name VARCHAR(150) NOT NULL,
  quantity INT NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(id)
);
```

Payment database

```
CREATE DATABASE payment_db;
USE payment_db;

CREATE TABLE payments (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_id BIGINT NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  payment_status VARCHAR(20) DEFAULT 'PENDING',
  transaction_reference VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```